Biomedical Data Processing: final project report

ECG-based identification and EEG-based sleep staging

Teodora Taleska

August 2026

1 Part I: ECG-based identification

The aim of this part is to recover, without supervision, which of five subjects produced each heartbeat in a single continuous ECG recording. The pipeline has six stages: R-peak detection with Pan-Tompkins, Wiener filtering (a PQRST model as the desired signal, the inter-beat intervals as the noise), segmentation into one-beat windows, wavelet decomposition, K-means clustering into five groups, and PCA followed by re-clustering to measure the cost of the compression. The ground-truth labels are never used inside this chain, only to score the finished partition, and since K-means numbers its clusters arbitrarily, the labelling is resolved afterwards by searching over all permutations of the five indices.

1.1 Data exploration

The recording is a single ECG channel sampled at 200 Hz, 121006 samples, and contains the heartbeats of five subjects who touch the sensor in turn, one beat at a time. Figure 1 shows a 20 s window of the unfiltered signal.

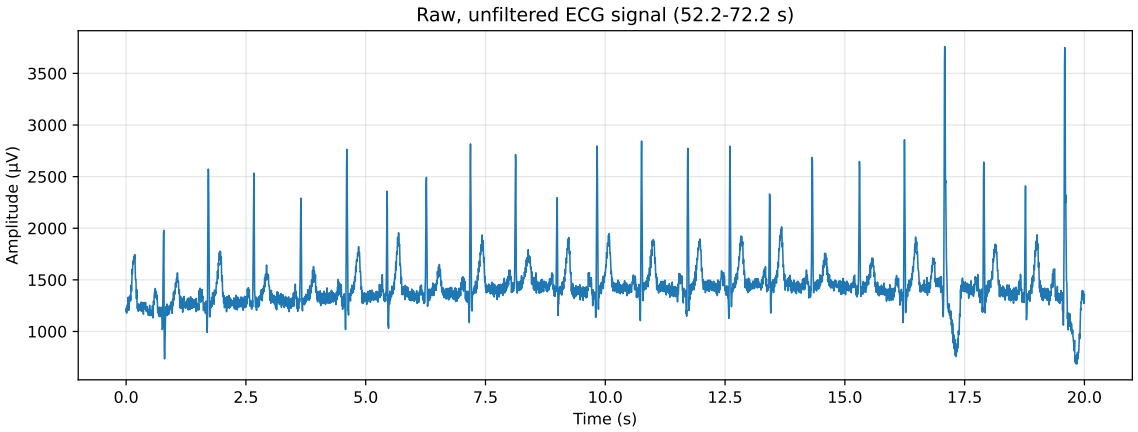


Figure 1: Twenty seconds of the raw, unfiltered ECG recording (52.2–72.2 s), with beats from several of the five subjects visible in turn.

Q: Which noise sources are present, and why? Three sources are visible on this plot.

Baseline wander.
The line drifts slowly up and down across the window instead of staying flat, far too slow to be cardiac activity. It tracks the electrode-skin contact, which changes with each subject: the clearest dip in the window coincides with a change of subject and does not have the shape of a PQRST complex.

Powerline interference.
A regular ripple with a fairly constant amplitude is visible throughout the signal, including the flat segments between heartbeats where no cardiac activity is expected. Its periodic nature and consistent appearance suggest that it originates from electrical mains interference rather than the ECG itself.

Broadband muscle and contact noise.
In addition to the periodic ripple, small irregular fluctuations can be observed on the signal, particularly in the flatter regions between heartbeats. These non-periodic variations are consistent with muscle activity, electrode movement, or other measurement-related noise.

1.2 Wiener filter

The recording is cleaned with a Wiener filter built from an averaged PQRST complex as the desired signal and the concatenated inter-beat intervals as the noise, then applied to the whole record with weighted overlap-add filtering.

1.2.1 Template estimation

R-peaks are located with the Pan-Tompkins algorithm. The signal is band-pass filtered between 5 Hz and 11 Hz using two cascaded fourth-order Butterworth stages, differentiated with a five-point derivative, squared, and integrated with a 150 ms moving window; peaks are then picked on this integrated signal.

Design choices. The assignment does not specify how to pick the peaks once the signal is integrated, so two parameters had to be chosen. The threshold is set to exactly the mean of the integrated signal (a factor of 1), which makes it a data-driven choice rather than a fixed value picked by hand: between beats the integrated signal reflects only noise and stays close to its own average, while a QRS complex produces a burst of energy well above that average, so thresholding at the mean separates the two without needing a value tuned to this particular recording's amplitude scale. The minimum spacing enforced between accepted peaks is set to 250 ms, a 240 bpm ceiling, as noted in the code comment next to it. This sits safely above any physiologically realistic heart rate, so it can never suppress two genuinely consecutive beats, while still being wide enough to span the 150 ms moving-window integration, which is what stops the integrated bump from a single beat from being picked up as two separate peaks.

1.2.2 Creating and applying the Wiener filter

Q: How did you compute the Wiener filter? What methods did you use for the PSD estimation? The Wiener filter is designed in the frequency domain as

$$W(f) = \frac{S_d(f)}{S_d(f) + S_n(f)},$$

with the average PQRST complex as the model of the desired signal d and the concatenated inter-beat intervals as the model of the noise n. Both signals have their mean removed before estimating their power spectral densities. The two spectra are not estimated the same way. S_n is estimated with Welch's method (Hann window, segment length equal to the filter length L, 50 % overlap), because the noise record is long and stationary and averaging over overlapping segments is what keeps the variance of the estimate down. S_d, however, is estimated as the periodogram of the zero-padded template rather than with Welch. This is a correction with respect to the first exam period, where both spectra were estimated with Welch: d is a single deterministic transient, not a stationary process, so splitting it into sub-windows and averaging them, as Welch does, mixes together shifted, non-stationary pieces of the same pulse and broadens its spectrum. The periodogram of the zero-padded template is the estimator that matches what d actually is. The filter's time-domain impulse response is the inverse FFT of $W(f)$, and it is applied to the full recording with weighted overlap-add (WOLA) filtering rather than direct convolution, which avoids having to filter 121 006 samples in one block.

Q: What length did you choose for the Wiener filter, and why? The template is 140 samples long (0.7 s). The filter length is set to $L = 512$ samples (2.56 s), about 3.7 times the template length, which leaves enough zero-padding to resolve the periodogram of S_d at a fine frequency resolution ($\mathrm{d}f = f_s/L \approx 0.39\,Hz$) without truncating any of the template's spectral content.

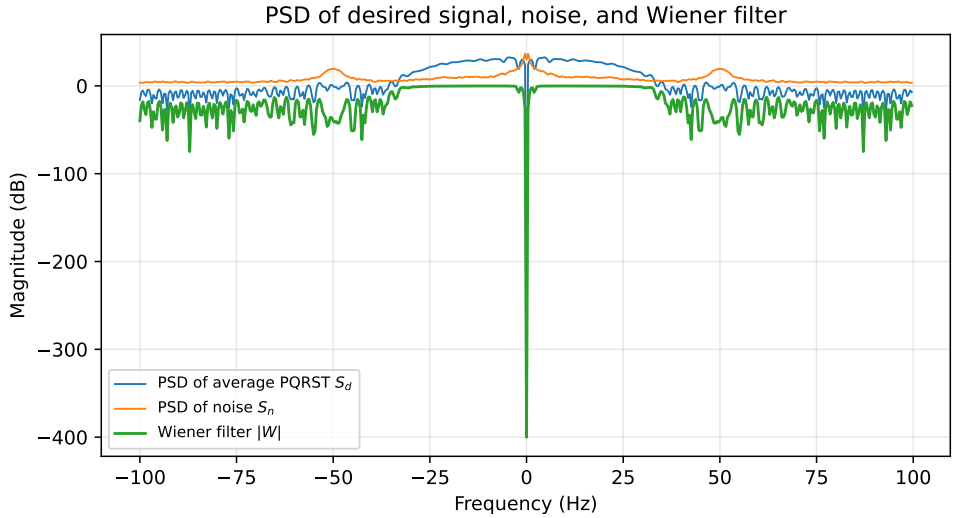


Figure 2: PSD of the average PQRST complex (S_d), PSD of the noise (S_n), and the resulting Wiener filter magnitude $|W|$, all in dB.

Figure 2 tracks the three noise sources identified in identified in Section 1.1. It is near zero at 0 Hz, where demeaning forces $S_d(0) = 0$ while $S_n(0)$ still carries the baseline wander; it stays close to unity gain across the band where the ECG carries most of its energy, since $W(f) = S_d/(S_d + S_n)$ can never exceed 0 dB; and it dips sharply at exactly 50 Hz and rolls off further above it, suppressing the mains line and the broadband muscle/contact noise identified earlier.

Q: After filtering, recompute the R-peak locations. Do you get the same number of R-peaks? Why (not)?

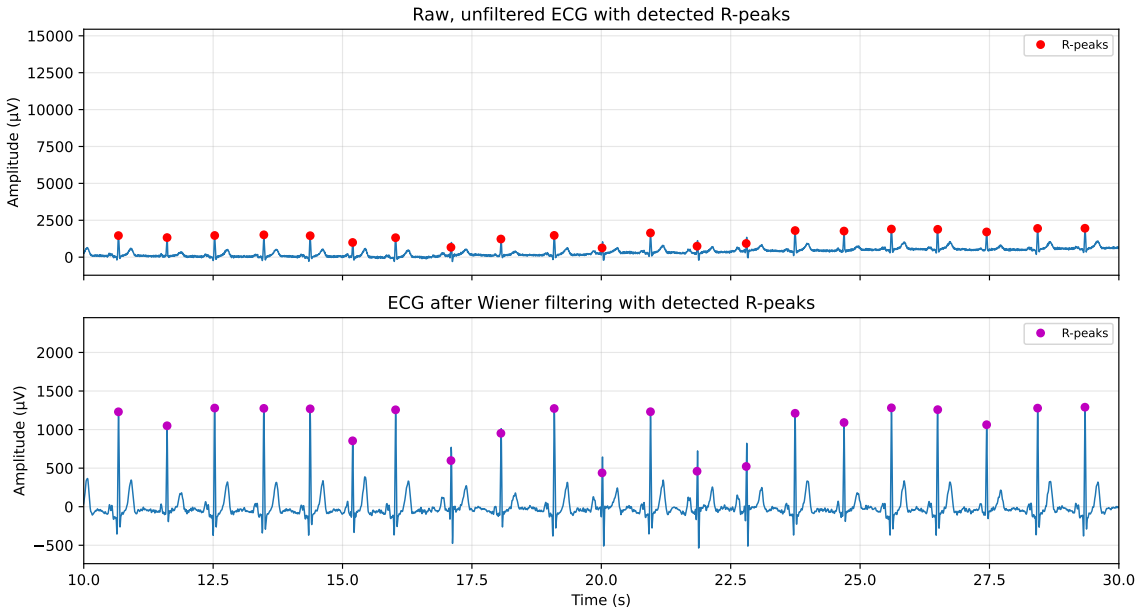


Figure 3: The same 20 s window before and after Wiener filtering, with the R-peaks detected by Pan-Tompkins on each version.

Pan-Tompkins is re-run on the filtered signal because the detector itself depends on filtering (its own internal band-pass and derivative stages), so a cleaner input can change which local maxima cross the threshold. Detection gives 679 peaks on the raw signal and 676 on the

filtered one, not the same count. As an independent check, the ground-truth labels give 511 uninterrupted contact runs with a median duration of 0.94 s; dividing each run's duration by that median and rounding to the nearest whole number of beats (at least one) gives an estimate of about 675 true heartbeats, close to both detector counts. The small difference between the raw and filtered counts is consistent with how `wola_filt` behaves at the boundaries of the recording: its analysis windows overlap by design, but the first and last windows have no neighbour to overlap with on one side, and the function zero-pads the reconstructed signal whenever it falls short of the original length. Beats sitting in these boundary regions are the ones most likely to be reconstructed poorly enough for Pan-Tompkins to miss, while the interior of the recording is covered by fully overlapping windows and filtered normally. This localised boundary behaviour of the overlap-add procedure, rather than a general degradation of the signal, is the most likely explanation for the three fewer peaks found after filtering.

1.2.3 Additional questions

Q: Assuming independent additive noise, should you consider the average PQRST complex as a template for the clean signal, or as a template for the signal with noise? Why? It is a template for the clean signal. Under this assumption every measured beat can be written as $x_i[t] = s[t] + n_i[t]$, with $s[t]$ the true PQRST waveform and $n_i[t]$ zero-mean noise that varies from beat to beat. Averaging many aligned complexes cancels the noise terms while leaving the shared signal component untouched, so the result is a noise-reduced estimate of $s[t]$, not of the noisy signal.

Q: Should you remove the mean from the PQRST complexes before averaging them? What about the mean of the noise segments? Yes, in both cases. The mean of the template carries no information about its shape and would otherwise appear as a large DC component in S_d that has nothing to do with ECG morphology; Wiener theory also assumes zero-mean noise, so a leftover offset in the noise segments would inflate $S_n(0)$ for reasons unrelated to the actual noise process. The effect is directly visible in this recording: demeaning forces $S_d(0) = 0$ while $S_n(0)$ stays large because of the baseline wander identified earlier, which is exactly why the filter rejects 0 Hz almost completely.

Q: Which method and parameters did you choose to estimate the PSDs? Why? S_n is estimated with Welch's method (Hann window, segment length $L = 512$, 50 % overlap), because the noise record is long (about 130 s) and stationary, and averaging over overlapping segments is what reduces the variance of the estimate; a single periodogram of a random signal has a relative standard deviation of about 100 % regardless of record length. S_d is estimated differently, as the periodogram of the zero-padded template rather than with Welch, since d is a single deterministic transient rather than a stationary process and averaging sub-windows of it would broaden its spectrum instead of resolving it.

Q: Do you see any edge effects after filtering? Why (not)? A small edge effect can be observed at the very beginning and end of the recording, which is expected due to boundary effects introduced by the filtering process. The `wola_filt` function reconstructs the filtered signal from overlapping analysis windows and explicitly zero-pads the result whenever the reconstruction falls short of the input length; the first and last windows also lack a neighbour to overlap with on one side. Both are boundary artefacts of the overlap-add procedure itself, rather than of the Wiener filter's frequency response, and are the most likely explanation for why the filtered signal yields three fewer detected R-peaks than the raw one.

1.3 Segmentation

The filtered signal is cut into one-beat segments around each R-peak (0.3 s before to 0.4 s after, as specified), and the ground-truth labels are segmented the same way, with each segment assigned the majority label of its samples. Averaging the segments per subject and counting how many segments each subject contributes give the two deliverables below.

Subject	Count	Percentage (%)
1	117	17.31
2	147	21.75
3	40	5.92
4	115	17.01
5	257	38.02

Table 1: Percentage of segments belonging to each subject.

The five subjects are not represented equally: subject 5 contributes the most segments and subject 3 the fewest. I additionally overlaid all five averages on one set of axes, shown in Figure 4.

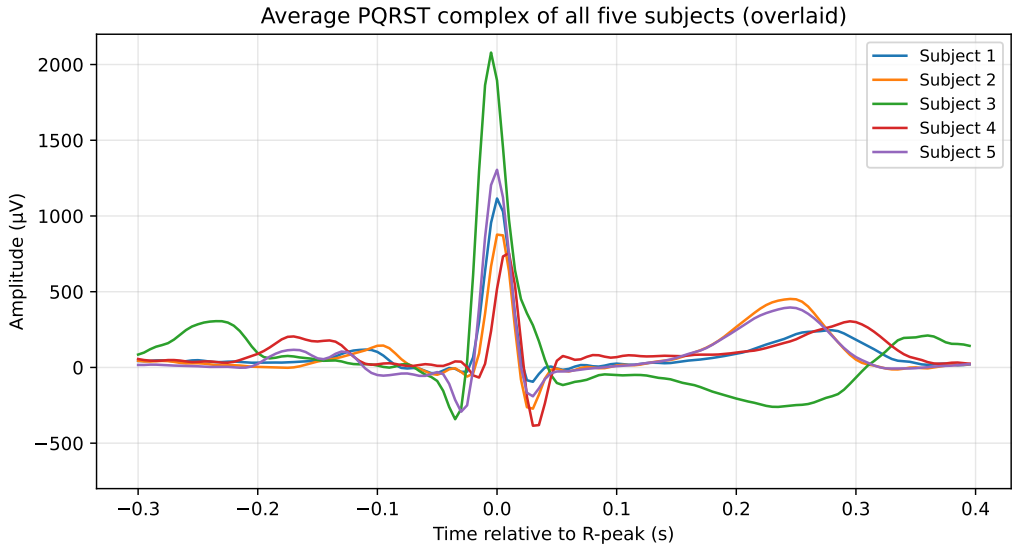


Figure 4: Average PQRST complex of all five subjects, overlaid on shared axes.

1.3.1 Additional questions

Q: Do you see any defining differences between the average PQRST complex of each subject? Yes. All five averages share the same overall structure and are aligned at the R-peak, but they differ in QRS amplitude, width and the morphology of the surrounding waves. Subject 3 is the clearest case: besides the taller R-peak, its average complex stays negative where the others show a positive wave, consistent with the valvular heart disease noted for that subject in the assignment description. The remaining four subjects vary more moderately among themselves in amplitude and width, without such a qualitative change in shape.

1.4 Wavelet decomposition

Each 0.7 s segment is decomposed with a level-5 Daubechies-4 (db4) wavelet transform, giving six coefficient arrays per segment (cA5 through cD1, 11+11+ 15+23+40+73=173 values in total), concatenated into one 173-value feature vector per beat. It is this feature vector, not the raw 140-sample segment, that is used as input to the clustering algorithm in the next section.

Q: Select two wavelet coefficients that you consider most important. Visualize them in a two-dimensional space, coloured by ground-truth label. Importance is measured as variance across the 676 segments, after first discarding coefficients whose variance is effectively zero (a `VarianceThreshold` step): those carry no information regardless of how the ranking is done, so they are removed before ranking rather than left to compete with genuinely informative coefficients. Among what remains, the two with the highest variance are kept: `cA5[8]` and

`cA5[10]`, both from the coarsest approximation band rather than from any detail band. That is not a coincidence: `cA5` describes the broad shape and amplitude of the whole beat, and it is at that scale, not in the fast, fine detail, that the subjects differ most, matching Section 1.3's finding that subject 3's average complex differs from the others mainly in overall amplitude and shape.

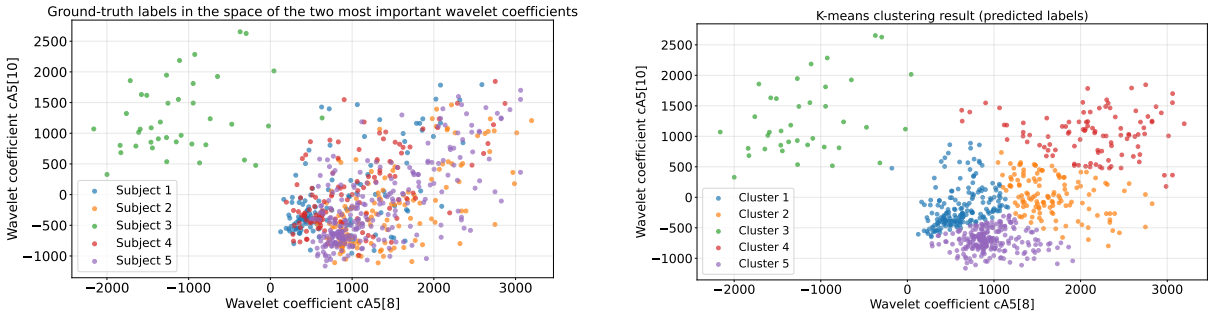
Plotted in this space (Figure 5 in Section 1.5, left panel), subject 3 is clearly separated from the other four along this pair of coordinates. The remaining four subjects overlap substantially with each other, which is expected from the selection method: ranking by variance alone favours whichever coefficients vary the most across the whole dataset, and that is driven by the subject that differs most overall (subject 3), without any guarantee that the other four are pulled apart from one another too.

1.4.1 Additional questions

Q: What is the advantage of this wavelet decomposition before clustering, compared to simply clustering the ECG segments directly? The decomposition does not shrink the number of values, 173 coefficients from a 140-sample segment is not a smaller representation, but it separates the beat into components at different scales instead of leaving it as a flat sequence of raw time samples. That separation is what the coefficient-selection step above relies on: the two coefficients that best set subject 3 apart both turned out to sit in the coarsest band, the one describing overall shape and amplitude, while the fine-detail bands contribute comparatively little to that particular difference. Clustering directly on raw samples has no equivalent way to tell a broad shape difference apart from fine timing jitter, since both simply show up as a pointwise difference at some sample index; the wavelet decomposition keeps that distinction explicit and available to work with.

1.5 K-means clustering

K-means with $k = 5$ is fit on the same two coefficients selected in Section 1.4 (`cA5[8]` and `cA5[10]`, z-scored first), and `match_labels` permutes the resulting cluster indices to whichever assignment best agrees with the ground-truth subjects, since K-means itself has no notion of which cluster index corresponds to which subject.



(a) Ground-truth labels (Section 1.4)

(b) K-means clusters, matched to ground truth

Figure 5: Ground-truth labels versus K-means clusters in the space of the same two coefficients, `cA5[8]` and `cA5[10]`.

Accuracy after matching is 0.420: subject 3 is recovered almost perfectly, matching the well-separated green cluster in Figure 5, while subjects 1, 2 and 5 spread across several columns instead of concentrating on the diagonal because they occupy the same overlapping region in this two-coefficient space, where K-means still draws three arbitrary boundaries since it assigns every point to its nearest centroid regardless of whether the classes are actually separable there; the confusion matrix is shown alongside the post-reduction one for comparison in Section 1.6.

Q: Did you normalise the wavelet coefficients before classification (without dimensionality reduction)? Yes. Both coefficients are z-scored with `StandardScaler` before K-means is

fit, so that neither one dominates the Euclidean distance purely because of its numerical range. In this particular pair the two coefficients' variances, 711700.64 for `cA5[8]` and 516391.97 for `cA5[10]` (Section 1.4), are already the same order of magnitude, so normalisation mainly enforces the general principle here rather than correcting a large scale mismatch; it is applied regardless, since relying on the two selected coefficients happening to be on comparable scales would not generalise to a different pair.

1.5.1 Additional questions

Q: Is the result of the unsupervised clustering always the same? Why (not)? Not in general. K-means starts from randomly initialised centroids, and different initialisations can converge to different local minima, particularly where classes overlap as they do here for subjects 1, 2 and 5. In this notebook the result is reproducible from run to run only because `random_state=42` fixes that randomness; without a fixed seed, the exact cluster boundaries through the overlapping region could come out differently each time, even though the well-separated subject 3 cluster would likely be recovered consistently regardless.

Q: Why do the names of the cluster labels not always match the names of the class labels? K-means has no access to the ground-truth labels and numbers its clusters arbitrarily, in whatever order it happens to initialise or converge. Cluster "1" has no reason to correspond to subject 1. `match_labels` searches over all permutations of the five indices and keeps whichever relabelling maximises agreement with the ground truth, which is what makes the accuracy and confusion matrix above meaningful.

Q: Is it a good idea to normalise the wavelets before classification? Why (not)? Yes, as a general rule, since K-means clusters purely by Euclidean distance, a coefficient with a larger numerical range would automatically carry more weight in that distance than a more informative coefficient on a smaller scale.

Q: How do you decide which wavelet values are the most relevant to model the ECG segments? Why? As in Section 1.4, relevance is measured as variance across the 676 segments after discarding near-constant coefficients, and the two coefficients kept here are the same highest-variance pair found there, `cA5[8]` and `cA5[10]`, both from the coarsest approximation band, since that is the scale at which the subjects differ most in overall shape and amplitude.

1.6 Dimensionality reduction

The wavelet features are reduced with PCA, fit on the full 173-coefficient set `X_wav` (shape 676×173) after z-scoring it with `StandardScaler`, the same order (normalise first, then reduce) as motivated in Section 1.4: PCA looks for directions of maximum variance, so without normalising first it would be dominated by whichever coefficients happen to have the largest raw scale, the coarse approximation band again, rather than by whichever directions are actually most informative. The number of components is chosen as the smallest number that reaches 90 % cumulative explained variance, a standard, simple stopping rule that does not require deciding on a target dimensionality by hand; on this dataset that threshold is reached at $N = 40$ components, a real reduction from 173 but far more than the 2 coefficients used in Section 1.5. For the 2D visualisation, the two dimensions plotted are the first two principal components, `PC1` and `PC2`: PCA already orders its components by variance, so these are "the two most relevant dimensions" under the same variance criterion used throughout this part, with no separate selection step needed.

Accuracy rises from 0.420118 before dimensionality reduction to 1.000000 after it with PCA, even though this step is nominally a *reduction*, because what is being compared is not "more dimensions vs. fewer" but "2 manually picked raw coefficients vs. 40 dimensions distilled from the entire 173-coefficient representation": the PCA step still uses far more of the original information than the two-coefficient visualisation in Section 1.5 ever did, the "reduction" is only relative to the full 173, not to the 2 used before.

Before reduction

$$\begin{pmatrix} 66 & 13 & 0 & 19 & 19 \\ 16 & 41 & 0 & 23 & 67 \\ 1 & 0 & 38 & 1 & 0 \\ 57 & 19 & 0 & 20 & 19 \\ 44 & 58 & 0 & 36 & 119 \end{pmatrix}$$

After reduction (PCA, $N = 40$)

$$\begin{pmatrix} 117 & 0 & 0 & 0 & 0 \\ 0 & 147 & 0 & 0 & 0 \\ 0 & 0 & 40 & 0 & 0 \\ 0 & 0 & 0 & 115 & 0 \\ 0 & 0 & 0 & 0 & 257 \end{pmatrix}$$

Table 2: Confusion matrices before and after dimensionality reduction.

Q: How did you reduce the feature dimensionality as much as possible? Did you use any normalisation? If yes, when? As described above: PCA on the full 173-value wavelet feature vector, keeping the smallest number of components that explains at least 90 % of the variance ($N = 40$), with the `StandardScaler` normalisation applied before PCA, not after.

1.6.1 Additional questions

Q: Should you normalise before dimensionality reduction? What about after dimensionality reduction? Yes before: PCA is sensitive to feature scale and would otherwise be dominated by whichever raw coefficients have the largest numerical range, not necessarily the most informative ones. Not after: PCA already orders the kept components by how much variance they explain on the normalised input, so re-normalising afterwards would force components that are not equally informative to contribute equally to any distance computed on them, undoing that ordering rather than preserving it.

2 Part II: EEG-based sleep staging

This part classifies 30 s epochs of a polysomnography (PSG) recording into one of five sleep stages (Wake, N1, N2, N3, REM) from four channels: two EEG derivations (Fpz-Cz, Pz-Oz), an EOG reference and an EMG reference. Twenty subjects are provided, split into fifteen for training and five held out for testing, and each subject's recording is already cut into consecutive 3000-sample epochs (100 Hz for 30 s) with one ground-truth stage label per epoch.

2.1 Data preparation

This first subsection groups the tasks that come before any feature is extracted: loading the recordings, visualising a single epoch, and removing EOG/EMG artefacts with ICA.

2.1.1 Visualising a single epoch

`plot_epoch` plots the four channels of one epoch stacked on a shared 30 s time axis. One epoch of each stage is located and plotted for subject 6: epoch 5 (Wake), epoch 64 (N1), epoch 74 (N2), epoch 173 (N3) and epoch 255 (REM), giving one concrete example of each stage to discuss.

Q: What visible patterns characterise each sleep stage?

Wake (subject 6, epoch 5).
The EEG channels show fast, low-amplitude activity, consistent with an active brain. The EOG shows small eye movements, and the EMG sits at a comparatively higher, more active level than in any of the sleep stages below.

N1 (subject 6, epoch 64).
The lightest sleep stage: the EEG slows down and amplitude stays low, without a single strong visual marker the way the later stages have. The EOG shows slow, rolling eye movements, and the EMG starts to relax relative to wake.

N2 (subject 6, epoch 74, Figure 6).
The stage with the clearest visual markers of all five: short bursts of faster oscillation standing out against a calmer background (sleep spindles), and isolated sharp, high-amplitude

Figure 6: Subject 6, epoch 74, an N2 epoch: Fpz-Cz, Pz-Oz, EOG and EMG, top to bottom.

deflections that stand out from the surrounding waveform (K-complexes). Both are best looked for in the two EEG channels at the top of Figure 6. The EOG is calm and the EMG lower than in N1.

N3 (subject 6, epoch 173).
Deep sleep: the EEG is dominated by high-amplitude, low-frequency waves, slower than anything in the stages above. The EOG is quiet and the EMG is at its lowest of the four non-REM stages.

REM (subject 6, epoch 255).
The EEG returns to a mixed, comparatively low-amplitude pattern that looks closer to N1 or wake than to N2 or N3, which is why the reference channels matter here: the EOG shows the rapid eye movements that give the stage its name, while the EMG drops to its lowest level of all five stages, consistent with the muscle atonia of REM sleep.

2.1.2 Removal of EOG/EMG artefacts from EEG

A FastICA model is fit on the training subjects (subjects 0–14) so that the two reference channels, EOG and EMG, can later be used to identify and remove the independent components they are associated with, leaving cleaner Fpz-Cz and Pz-Oz channels for feature extraction.

Q: How should you reshape the data properly? Each subject's array has shape (epochs, channels, samples), with 4 channels and 3000 samples per epoch (30 s at 100 Hz), but scikit-learn's `FastICA` expects samples-by-channels. The channel and sample axes are swapped first, so each epoch becomes (samples, channels), and the epoch and sample axes are then merged into one, so every 3000-sample epoch contributes 3000 rows to a single (samples, 4) array. Doing this for all 15 training subjects and stacking them gives a training set of shape `(47280000, 4)`.

Q: Which data were concatenated, and what is the final training shape? All epochs of subjects 0–14 are reshaped this way and stacked with `np.vstack`, giving the `(47280000, 4)` array above as the data `FastICA` is fit on. The model itself is built with 4 components (one per channel), `whiten='unit-variance'`, and a fixed `random_state`: FastICA starts from a random unmixing matrix and the sign and ordering of the recovered components are not

otherwise identified by the model, so fixing the seed is what makes the same four components come out on every run.

Visual inspection of ICA components. For the same subject and epoch used earlier (subject 6, epoch 5, Figure 6), the epoch is reshaped from (4, 3000) to (3000, 4) with a transpose so it matches the layout `ica.transform` expects, transformed into its four independent components, and transposed back to (4, 3000) so each component can be plotted as its own row against time. Figure 7 shows the four components next to the original EOG and EMG channels.

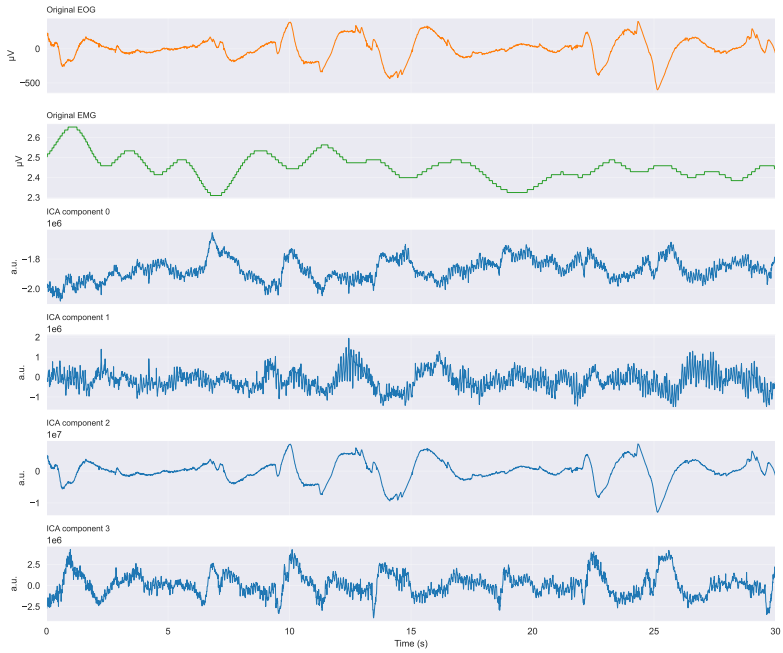


Figure 7: Original EOG and EMG channels (top two rows) alongside the four ICA components, subject 6, epoch 5.

Q: Identify which components are likely EOG-related and which are EMG-related. Provide a short rationale for each identification. Component 2 tracks the original EOG channel closely: both show the same slow, large deflections at roughly the same points in the epoch, and component 2's amplitude is an order of magnitude above the other three (10^7 against 10^6), consistent with the large, slow signal eye movements produce. Identifying the EMG-related component from the plot alone is less clear, since the original EMG channel is small and largely featureless over this particular epoch. Of the remaining three, component 1 shows the most irregular, high-frequency fluctuation, closer to what broadband muscle activity looks like than the smoother, slower content of components 0 and 3, which makes it the more likely EMG candidate from visual inspection.

Q: Discuss the effect of including vs excluding EOG/EMG channels from ICA input. Including EOG and EMG as explicit channels gives the ICA input direct access to the artefact sources themselves, so eye-movement and muscle activity are much more likely to end up isolated into their own independent components, components that can then be zeroed out and the mixing inverted to recover cleaner EEG. Excluding EOG/EMG would remove that direct signal and leave ICA to separate the same artefacts purely from how they leak into the two EEG channels, a weaker and less reliable separation.

Automatic ICA component elimination. `identify_artifact_components` takes the maximum-absolute-correlation rule as its deterministic selection rule, since ICA's whitened components have an arbitrary sign, so only the strength of the association with EOG or EMG is meaningful,

not its direction. The correlation is recomputed separately for every subject rather than fixing one global component index, so the rule keeps working even though the same fitted unmixing matrix is applied to all twenty subjects. Exactly one component is zeroed for EOG and one for EMG, which matches the four-channel mixture the model was fit on (two EEG channels plus the two reference channels): removing the two artefact components leaves exactly two components behind, and `denoise_subject_with_ica` returns the first two channels of the reconstructed signal, Fpz-Cz and Pz-Oz, in the channel order used throughout.

Q: Visualise the signal before and after applying ICA, with the original and de-mixed signal overlaid on the same axes for each channel. Figure 8 shows subject 6, epoch 5 (the same example used above) before and after the two identified components are removed, Fpz-Cz on top and Pz-Oz below. In both channels the large, slow deflections visible in the original signal are strongly attenuated in the denoised trace, leaving a much smaller, comparatively flat signal, consistent with the identified EOG/EMG components carrying most of that large-amplitude activity.

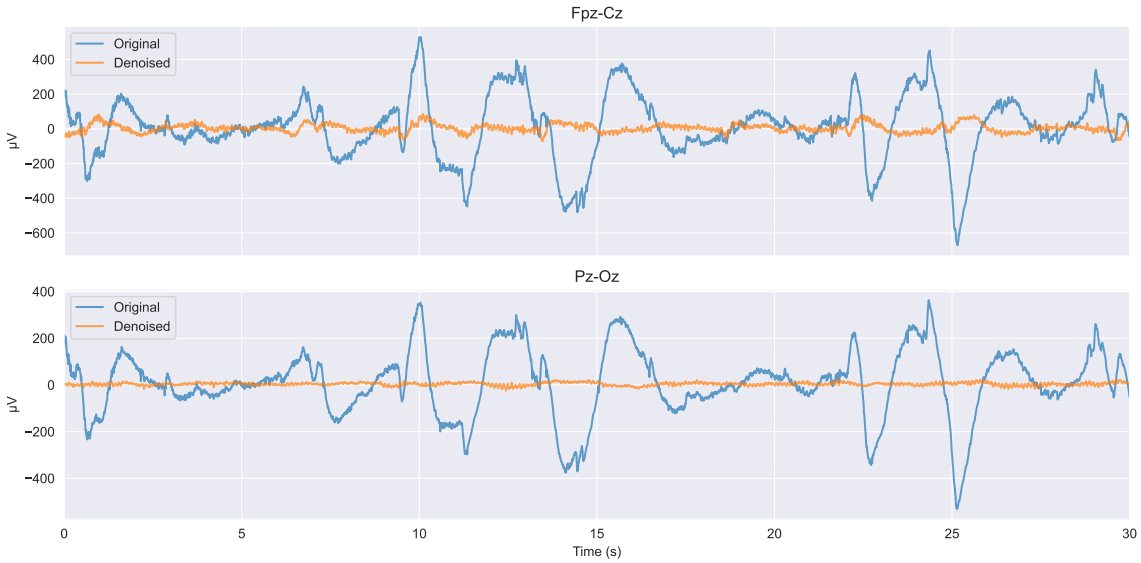


Figure 8: Fpz-Cz and Pz-Oz, subject 6, epoch 5, before and after removing the identified EOG/EMG components.

2.1.3 Signal filtering

The goal here was to design and apply a filter that preserves the EEG spectral content relevant for sleep staging while removing slow drifts and high-frequency noise. `bandpass_filter` is a 4th-order Butterworth bandpass, 0.5–30 Hz, built with `scipy.signal.butter(..., output="sos")` and applied with `sosfiltfilt`. The passband edges are not arbitrary: 0.5 Hz and 30 Hz are exactly the lower edge of the delta band and the upper edge of the beta band in the `BANDS` dictionary used later for feature extraction, so the filter keeps everything the band-power features are computed from and removes only what sits outside it, the slow drift below 0.5 Hz and high-frequency noise and residual EMG activity above 30 Hz. The second-order-sections form together with `sosfiltfilt` is used instead of a direct transfer-function implementation for numerical stability at this filter order, and `sosfiltfilt` filters forward and backward so the result has zero phase, which avoids shifting the timing of EEG waveforms such as the spindles and K-complexes discussed around Figure 6.

Figure 9 shows that most of the large-amplitude activity is already removed by ICA denoising, and filtering flattens the signal further into a smaller, smoother trace with the remaining slow wandering gone. Figure 10 confirms this in the frequency domain: the two curves track each other closely inside the passband, then the filtered curve falls away sharply above 30 Hz, several

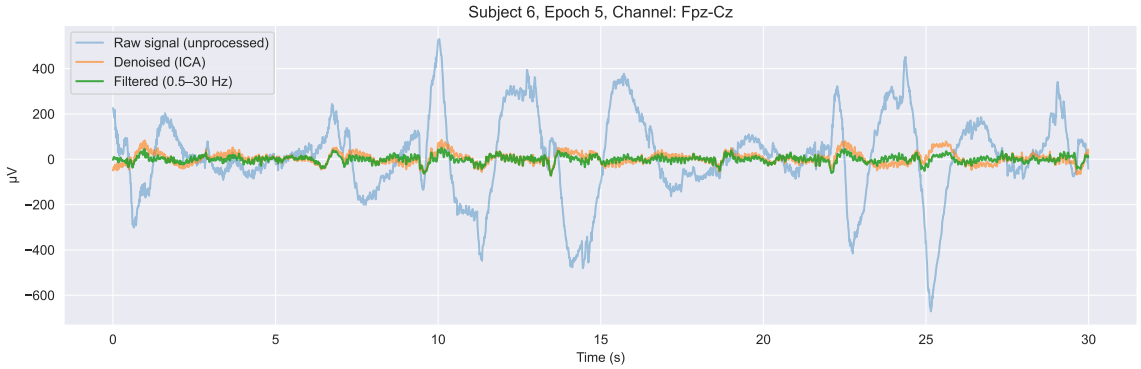


Figure 9: Subject 6, epoch 5, Fpz-Cz: raw, ICA-denoised, and denoised-and-filtered signal.

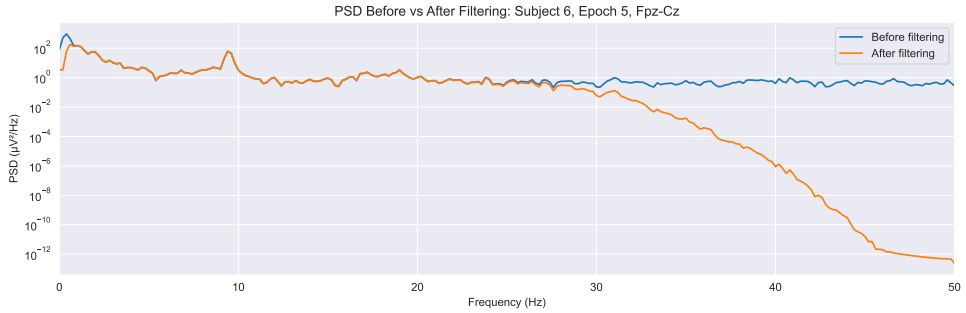


Figure 10: Welch PSD of the same epoch before and after filtering.

orders of magnitude below the unfiltered curve by 50 Hz, exactly the attenuation the low-pass edge is designed to produce.

Q: Should you filter before or after ICA artefact removal? ICA should run first, on the unfiltered signal. ICA assumes the four channels are linear mixtures of the same underlying sources, and filtering the signal beforehand would distort that mixing; eye blinks and muscle bursts spread their energy across a wide range of frequencies, so removing part of that range before ICA runs would take away exactly the information it needs to isolate them as separate components. The bandpass filter is applied afterwards, to the already denoised signal.

2.1.4 Feature extraction

Here the goal was to extract a compact set of spectral features from each epoch, suitable for supervised sleep staging.

`extract_psd_features` computes, for each EEG channel (Fpz-Cz and Pz-Oz) and every epoch of a subject, the Welch periodogram of the μV-scaled, ICA-denoised signal (`fs=100`, `nperseg=300`, 3 s segments, a frequency resolution of about 0.33 Hz), and converts the resulting power to decibels with `10*np.log10(Pxx + 1e-12)`, the small additive constant keeping the logarithm defined where the estimated power is essentially zero. The dB scale compresses the wide dynamic range of EEG power, large near 0 Hz and decaying with frequency as Figure 10 already showed, into a scale where proportional differences across the spectrum are easier to compare. For each subject the function returns one (n_epochs, n_freq_bins) array per channel and the shared frequency vector (151 bins for `nperseg=300`, up to the 50 Hz Nyquist frequency); stacking this over all twenty subjects gives `PSD_all`.

As the bonus feature, `extract_bandpower` reuses the same Welch PSD but integrates it, with the trapezoidal rule, over each of the five canonical bands already defined in `BANDS` (delta through beta), for the same two EEG channels, giving ten raw-power features (μV^2, not dB) per epoch, stacked over subjects into `BANDPOWER_all`. As the notebook's own answer explains, this bandpower summary is a compact, physiologically meaningful representation of an epoch, since

it reflects known per-stage characteristics directly, such as the increased delta power expected in N3.

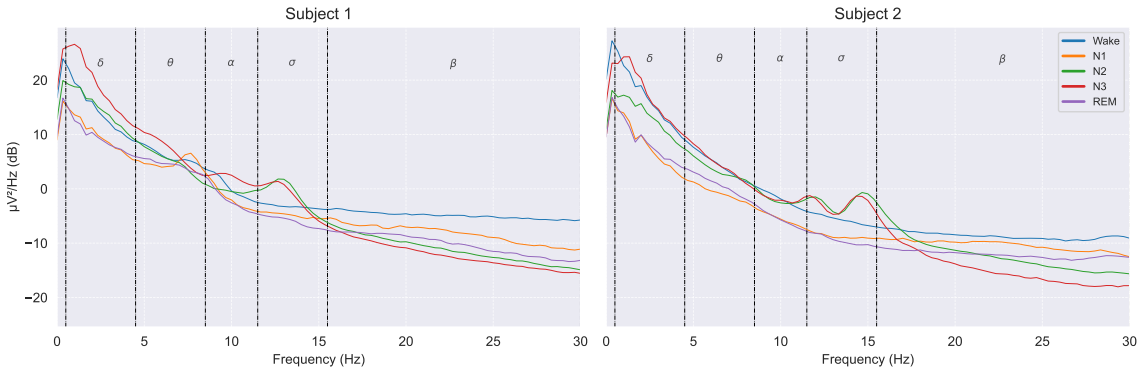


Figure 11: Mean Fpz-Cz PSD (dB) per sleep stage, subjects 1 and 2, using the provided `plot_subjects_features` helper.

Figure 11 shows the five stages separating most clearly in the low-frequency delta/theta range, and a distinct bump inside the sigma band for some stages, consistent with sleep-spindle activity concentrated in that band.

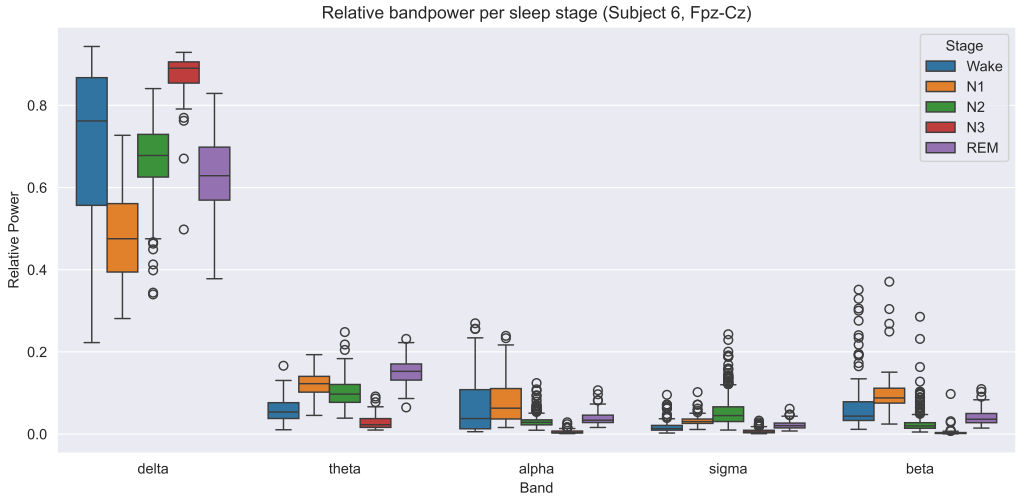


Figure 12: Relative bandpower per sleep stage, subject 6, Fpz-Cz.

For the box-plot summary of the bonus feature, each epoch's ten raw bandpower values are first normalised to relative power, before being grouped by band and stage. Figure 12 shows delta carrying most of the relative power in every stage, highest in N3 and lowest in N1, while the other four bands stay comparatively small and similar in size across stages, matching the delta-power characteristic the bandpower feature was added to capture.

Q: Is high/low/bandpass filtering needed if we remove the reduced frequencies afterwards during feature extraction? Why (not)? Yes, filtering is still necessary. Although PSD-based feature extraction focuses on specific frequency bands, an unfiltered signal still contains slow drifts and high-frequency noise that can leak into the estimated spectrum through windowing effects and the finite resolution of the Fourier transform. Band-pass filtering before PSD computation improves numerical stability, reduces spectral leakage, and makes the extracted features more accurately represent physiological EEG activity.

Q: Should other channels (EOG and/or EMG) be taken into consideration? No: as implemented, only the two EEG channels are used. Including EOG or EMG directly as staging

features risks reintroducing the same artefacts the ICA step was meant to remove from the EEG-based representation.

2.2 ML deployment

This section deploys a single classifier for sleep staging, chosen by a leave-one-subject-out (LOSO) cross-validation over the training subjects, and evaluated on the five held-out test subjects.

2.2.1 Scoring, classifiers and CV split

Scoring. Since this is a multi-class problem with imbalanced sleep stages, three metrics are tracked: accuracy, the overall proportion of correctly classified epochs; F1-macro, the F1-score averaged across the five stages unweighted, so rare stages such as N1 are not drowned out; and F1-weighted, the F1-score averaged with each stage weighted by its true frequency. Accuracy alone can be misleading here, since stages such as N2 are far more common than N1, so F1-macro and F1-weighted give two complementary views alongside it.

Preprocessing configurations. Six configurations are compared, listed in Table 3, covering the full range: no reduction at all, two manual reductions (integrating or averaging the PSD into per-band values), and automatic reduction with PCA at three component counts. Every configuration scales its features with `StandardScaler` first, including the tree-based models, for consistency across the comparison.

Table 3: Preprocessing configurations compared in the LOSO grid.

Configuration	Feature source	Reduction	Dim.
`psd_scaler`	PSD (Fpz-Cz), dB	none (scaling only)	151
`bandpower_scaler`	Bandpower, both channels	manual: trapz power per band	10
`psd_bandavg_scaler`	PSD (Fpz-Cz), dB	manual: mean PSD per band	5
`psd_pca10`	PSD (Fpz-Cz), dB	automatic: PCA	10
`psd_pca20`	PSD (Fpz-Cz), dB	automatic: PCA	20
`psd_pca30`	PSD (Fpz-Cz), dB	automatic: PCA	30

Classifiers. Five classifiers are compared, each swept over the hyperparameter grid in Table 4: 28 hyperparameter settings in total, each evaluated under all 6 preprocessing configurations, for 168 pipelines evaluated over the LOSO split.

Table 4: Classifiers and the hyperparameter grid swept for each.

Classifier	Hyperparameters swept	Settings
K-nearest neighbours	$n\_neighbors \in \{3, 5, 7, 9\}$	4
Logistic regression	$C \in \{0.1, 1.0, 10.0\}$, `max_iter`=1000	3
Decision tree	$max\_depth \in \{\text{None}, 10, 20\} \times min\_samples\_split \in \{2, 5, 10\}$	9
Random forest	$n\_estimators \in \{100, 200\} \times max\_depth \in \{\text{None}, 10, 20\}$	6
SVM (RBF kernel)	$C \in \{0.5, 1.0, 2.0\} \times gamma \in \{\text{scale}, 0.01\}$	6

LOSO split. `build_feature_matrix` stacks the epochs of a given list of subjects into one feature matrix and simultaneously builds a `groups` array that repeats each subject's ID once per epoch, so every row of the feature matrix carries the ID of the subject it came from. `loso_split_manual` then loops over the unique subject IDs in `groups` and, for each one, yields it as the validation fold (`groups == subj`) and every other subject as the training fold (`groups != subj`), which is exactly leave-one-subject-out: each fold's validation set is a single, whole subject, and no subject's epochs are ever split across training and validation within a fold. Only the fifteen training subjects go into this loop, since `X_psd_train`, `X_bp_train`, and

`X_psd_bandavg_train` are all built from `SUBJECTS_TRAIN`; the five test subjects are only turned into feature matrices later, after a classifier has already been selected, so they never take part in cross-validation.

2.2.2 Cross-validation and visualisation of results

Each of the 168 pipelines from Table 3 and Table 4 is passed once to `evaluate_loso`, which runs the 15-fold LOSO split and returns the mean and standard deviation of accuracy, F1-macro and F1-weighted across those folds. The deliverable suggests scikit-learn's `cross_validate`, but the notebook uses this hand-written function instead, since the subject-grouped folds and the three custom metrics are easiest to control directly with `loso_split_manual`.

Q: Which is the best model? Why? `cv_df["f1_macro_mean"].idxmax()` selects `psd_scaler` (the full 151-bin PSD, scaled, no dimensionality reduction) with an RBF-kernel SVM, $C = 1.0$, $\gamma =$ scale, F1-macro 0.678. F1-macro is the criterion because sleep staging is multi-class and imbalanced, and it is the one of the three tracked metrics that weights every stage equally, so a model cannot win by being good only at the frequent stages. Among the configurations compared, PSD features are the highest-dimensional input, and SVM suits that best: it looks for the maximum-margin boundary rather than relying on local distances or axis-aligned splits, which is why KNN and the tree-based models score visibly lower on the same features.

2.2.3 Deploying and testing the best model

The winning pipeline is rebuilt from fresh, unfitted steps and fit once on all fifteen training subjects combined, rather than reusing any single LOSO-fold model, since cross-validation is only used to pick the configuration. The confusion matrix uses `normalize="true"`, so each row sums to one and shows how a true stage's epochs were distributed across predictions, which matters given how unevenly sized the test stages are, from 264 epochs for N1 up to 2351 for N2.

Q: Are your results in line with your expectations? Yes. The model achieves strong performance on the dominant, well-defined stages, N2, N3 and Wake, while it struggles with N1, which is known to be difficult to predict since it lacks distinctive, easily characterised patterns; the notebook's classification report shows this plainly, N1's recall is only 0.034 against 0.812–0.853 for the other stages, and its confusion matrix shows most of N1's epochs predicted as REM or Wake instead. The generalisation gap between the LOSO cross-validation score (F1-macro 0.678) and the held-out test score (F1-macro 0.610) is moderate and expected for subject-independent sleep staging with classical ML models.

Q: Can anything be done to improve the model? Yes. Performance could improve by incorporating features from the EOG and EMG channels, which carry physiological information directly relevant to staging; including them could help the model separate stages that look similar in the EEG alone, particularly N1. Class-balanced learning would force the model to weight rare classes more, which should improve recall for underrepresented stages such as N1. Temporal smoothing across consecutive epochs, using the predictions of neighbouring epochs to correct isolated misclassifications, could produce more physiologically meaningful staging. Sequence-based models such as hidden Markov models could also help by learning typical sleep stage transitions directly and using that temporal context in the prediction, which should improve both accuracy and consistency.